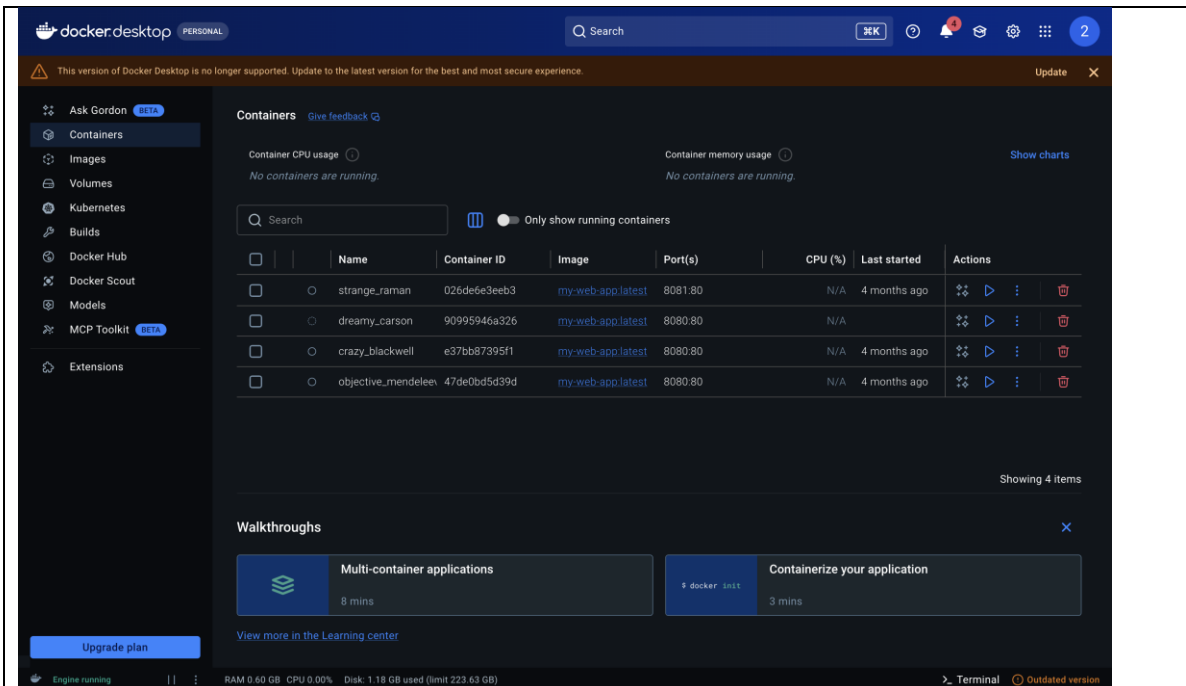


CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
CHANDUBHAI S. PATEL INSTITUTE OF TECHNOLOGY
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH
OCCE4001 - SOFTWARE TESTING

PRACTICAL LIST – AY: 2026- 27

Sr. no.	Title
4.	Title: CI/CD Pipeline Testing using Jenkins and Docker Aim: To automate software testing through a CI/CD pipeline using Jenkins and Docker. Software/Tools Required • Jenkins • Docker Desktop • GitHub • Selenium Test Script • Java/Python • Internet Connection Procedure 1. Install Jenkins and Docker. 2. Clone a sample project from GitHub. 3. Configure a Jenkins pipeline. 4. Execute Selenium test cases automatically. 5. Build and run the application inside a Docker container. 6. Verify the build status and test execution report. 7. Record the observations. Expected Outcome: Automated test execution is successfully integrated into a CI/CD pipeline. Result: CI/CD testing was successfully performed using Jenkins and Docker
Solution: Step 1: Install Docker Desktop and Verify Installation Download Docker Desktop, install it, sign in to your Docker account, and verify the installation using <code>docker --version</code>. https://www.docker.com/products/docker-desktop	

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
CHANDUBHAI S. PATEL INSTITUTE OF TECHNOLOGY
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH
OCCE4001 - SOFTWARE TESTING



Verify Docker install : `docker --version`

Step 2: Run Jenkins Container

Run Jenkins using Docker. If port 8080 is unavailable, recreate the container using port 8081.

`docker run -d -p 8080:8080 -p 50000:50000 --name jenkins jenkins/jenkins:its`

```
Last login: Mon Aug 3 12:31:51 on console
nishitprashantdesai@Nishits-MacBook-Air ~ % docker run -d -p 8080:8080 -p 50000:50000 --name jenkins jenkins/jenkins:its
Unable to find image 'jenkins/jenkins:its' locally
^[[Alts: Pulling from jenkins/jenkins
2829edc0fdb7: Pull complete
dc6208660f6d: Pull complete
61ffc360781a: Pull complete
6b89e501e8ef: Pull complete
5173f1cae393: Pull complete
ca6f6053557d: Pull complete
25154e0ed080: Pull complete
0aa498953a8c: Pull complete
83062cf32922: Pull complete
37418d21d488: Pull complete
c986c5ac35f9: Pull complete
f7d230c28692: Pull complete
0124ad05291d: Download complete
Digest: sha256:8547df3b0db2803d15ecc9499207a056bb30c23fddc18bb5b4a4dc14e77dd09
Status: Downloaded newer image for jenkins/jenkins:its
62d07d4d8c9798f6e04012323eb0a91800e2af9bfe51c8f867b12be2b0afeb58
nishitprashantdesai@Nishits-MacBook-Air ~ % █
```

Check if Jenkins container is running: `docker ps`

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
CHANDUBHAI S. PATEL INSTITUTE OF TECHNOLOGY
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH
OCCE4001 - SOFTWARE TESTING

If Container ID is blank then here jenkins is installed but 8080 port is not available
So First remove jenkins and change port. 8080 to 8081.

```
docker rm jenkins
```

```
docker run -d -p 8081:8080 -p 50000:50000 --name jenkins jenkins/jenkins:its
```

```
nishitprashantdesai@Nishits-MacBook-Air ~ % docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
62d07d4d8c97   jenkins/jenkins:its  "/usr/bin/tini -- /u..." 44 seconds ago Up 43 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp  jenkins
nishitprashantdesai@Nishits-MacBook-Air ~ %
```

Now It's perfect working

Step 3: Retrieve Jenkins Password

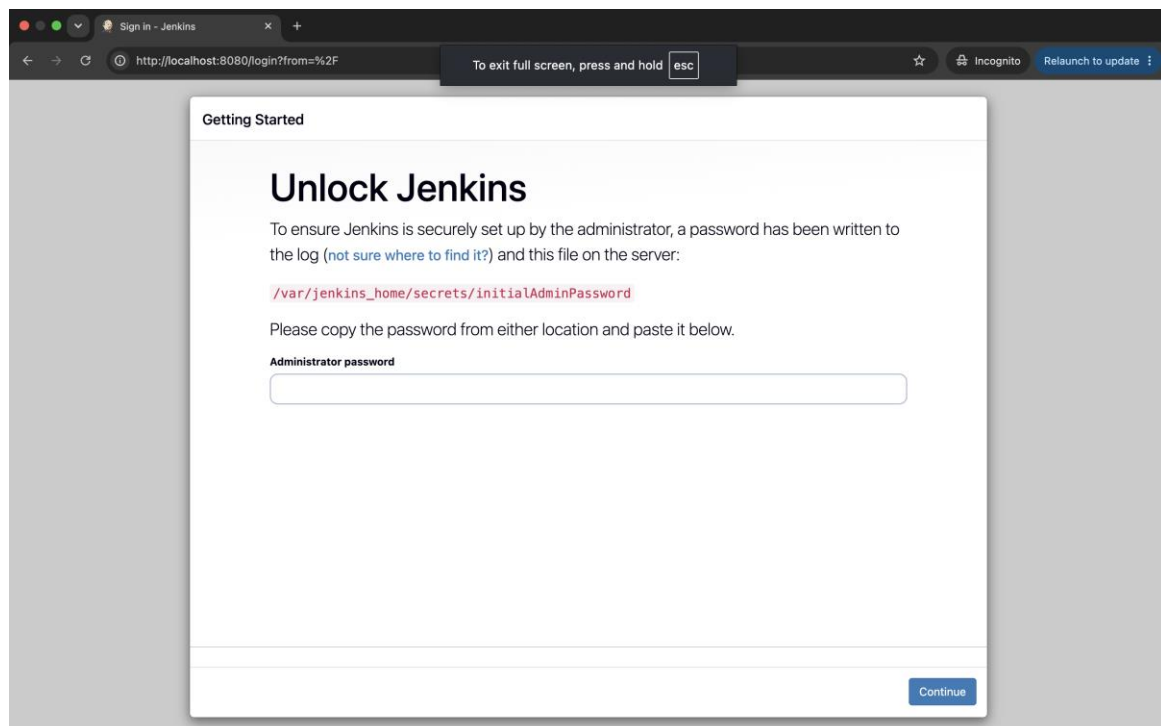
Retrieve the Jenkins initial administrator password and save it securely.

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Step 4: Unlock Jenkins

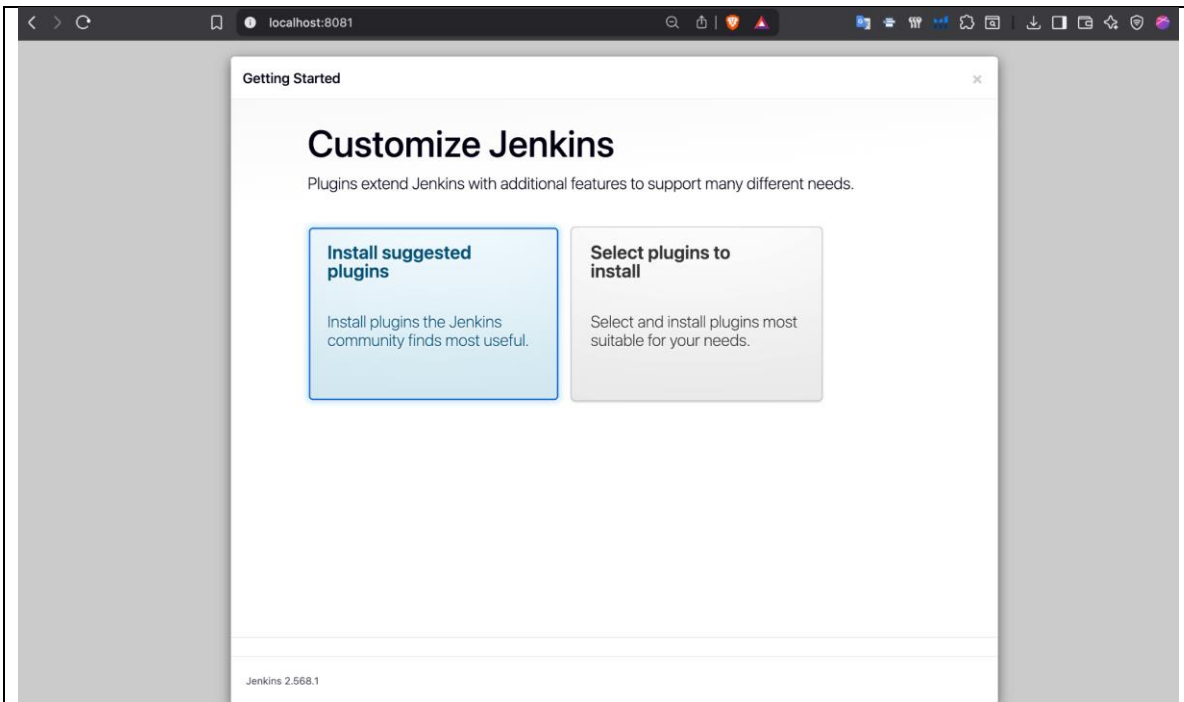
Open Jenkins in the browser and enter the administrator password.

Set up Jenkins in the browser with <http://localhost:8081> in browser



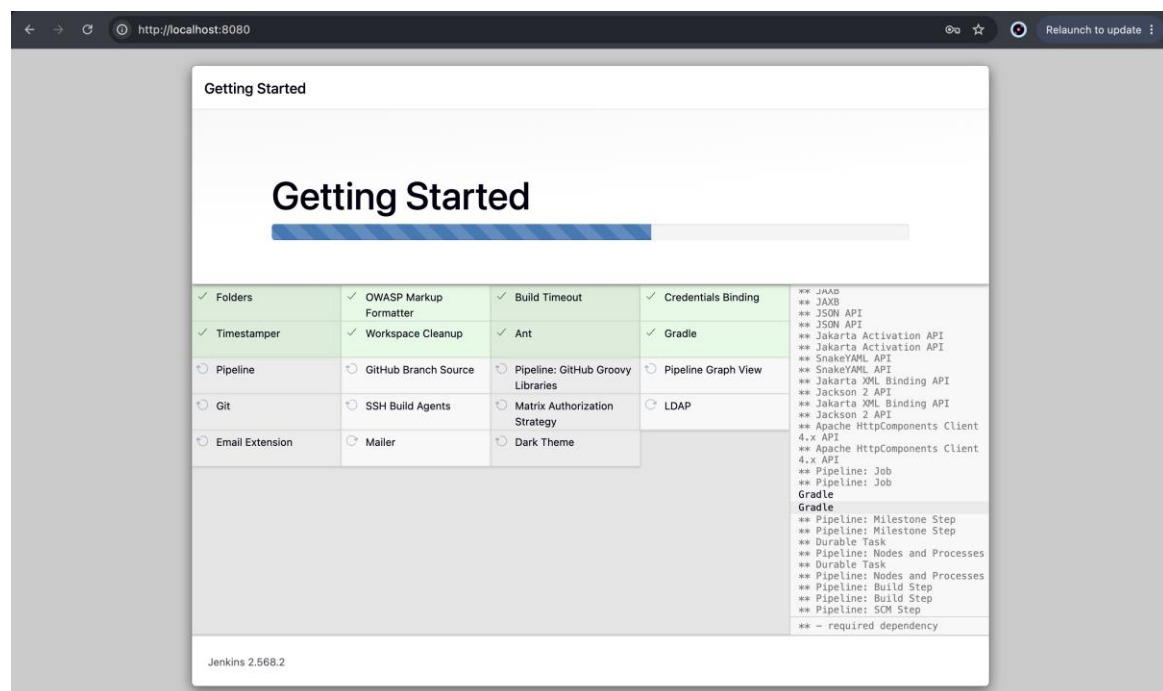
Enter password and Continue

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
CHANDUBHAI S. PATEL INSTITUTE OF TECHNOLOGY
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH
OCCE4001 - SOFTWARE TESTING



Step 5: Install Plugins

Install the suggested plugins and wait for the installation process to complete.

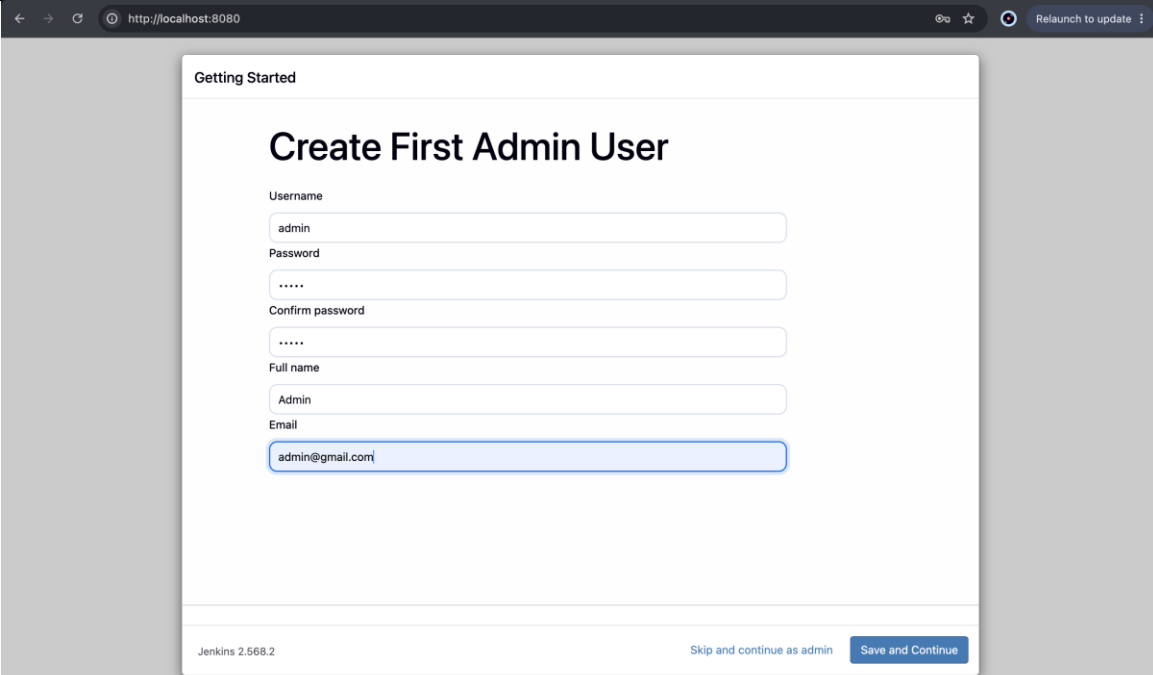


Wait until completion of the entire installation of all plugins and dependencies.

Step 6: Create Administrator Account

Create the first Jenkins administrator account and complete the setup wizard.

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
CHANDUBHAI S. PATEL INSTITUTE OF TECHNOLOGY
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH
OCCE4001 - SOFTWARE TESTING

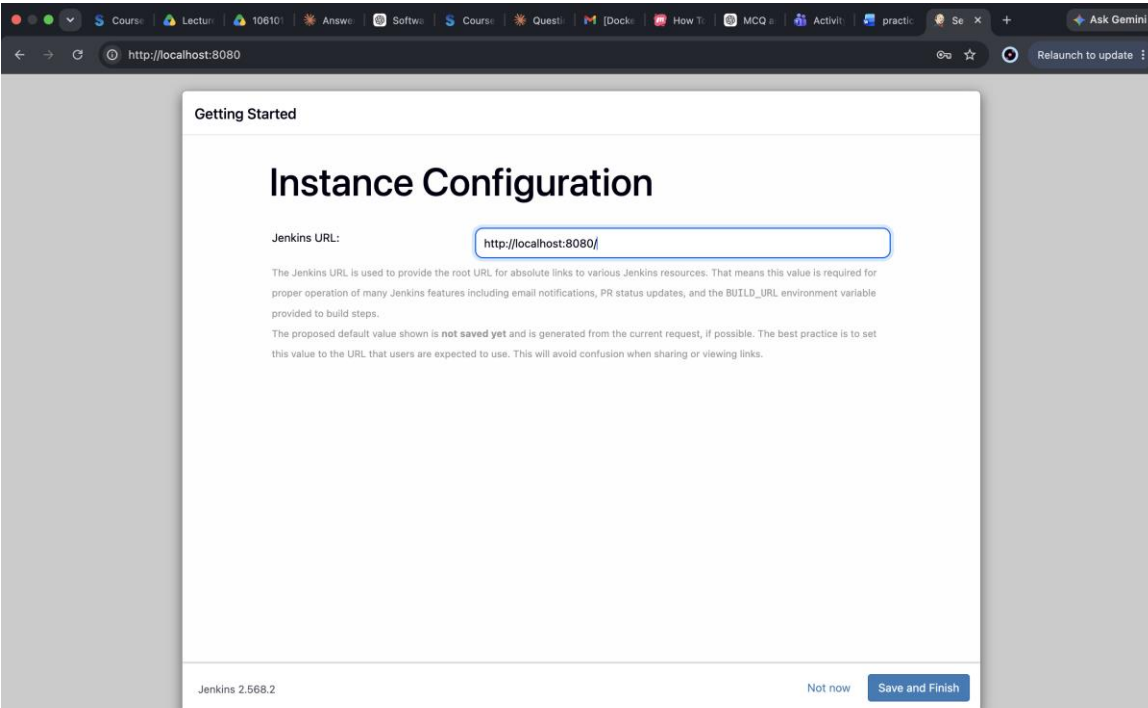


The screenshot shows the Jenkins 'Getting Started' page with the 'Create First Admin User' form. The form fields are filled with the following values:

- Username: admin
- Password:
- Confirm password:
- Full name: Admin
- Email: admin@gmail.com

At the bottom of the form, there are two buttons: 'Skip and continue as admin' and 'Save and Continue'. The Jenkins version 2.568.2 is displayed in the bottom left corner.

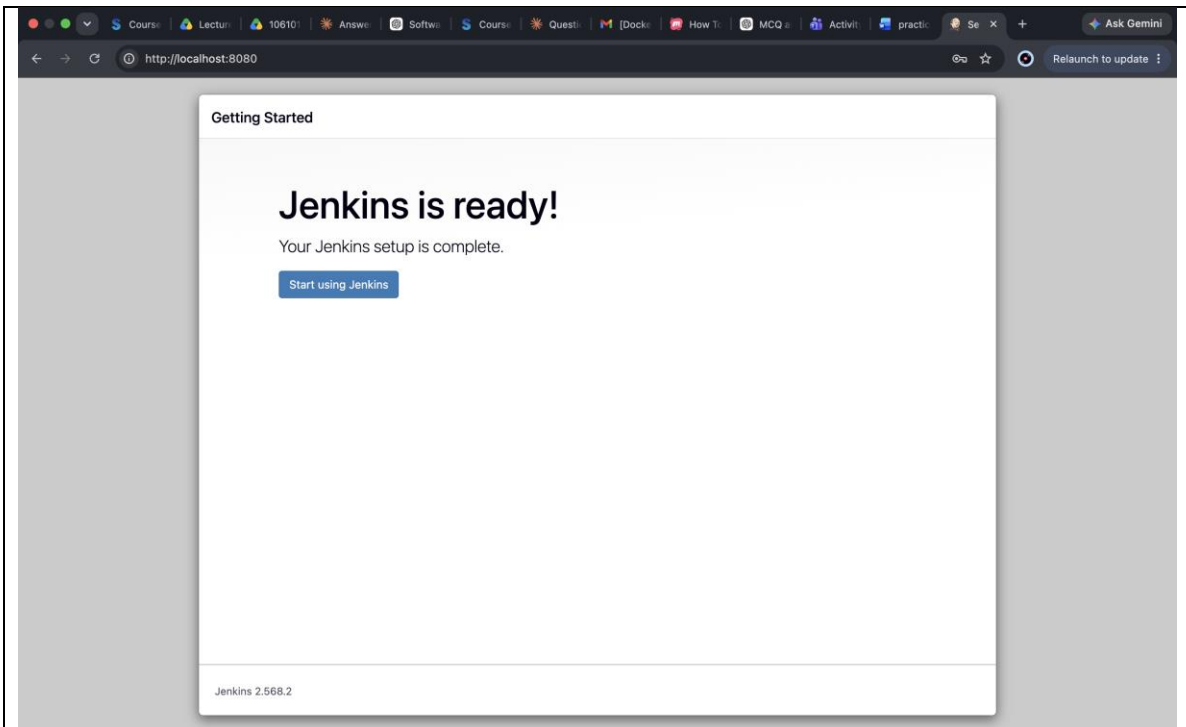
Enter required details and save & continue



The screenshot shows the Jenkins 'Getting Started' page with the 'Instance Configuration' form. The 'Jenkins URL' field is filled with the value 'http://localhost:8080/'. Below the field, there is a detailed explanation of the Jenkins URL and its importance for various features. At the bottom of the form, there are two buttons: 'Not now' and 'Save and Finish'. The Jenkins version 2.568.2 is displayed in the bottom left corner.

Save and finish.

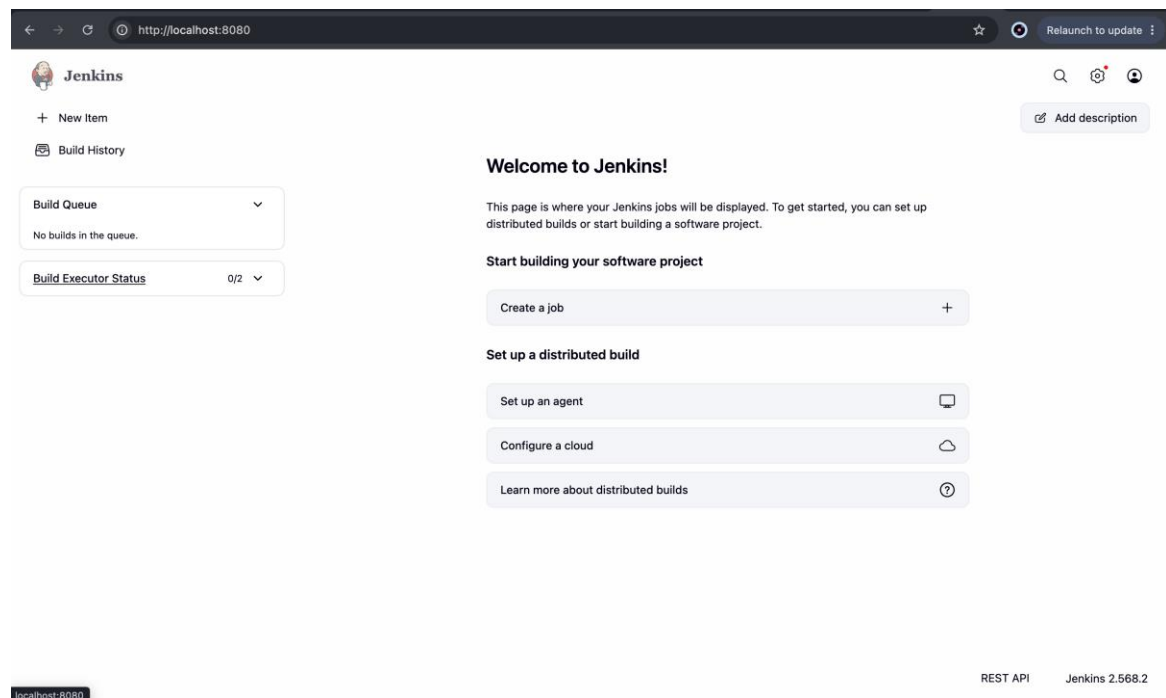
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
CHANDUBHAI S. PATEL INSTITUTE OF TECHNOLOGY
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH
OCCE4001 - SOFTWARE TESTING



Start using jenkins.

Step 7: Access Jenkins Dashboard

Start Jenkins and verify that the dashboard loads successfully.



Step 8: Create Project Files

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
CHANDUBHAI S. PATEL INSTITUTE OF TECHNOLOGY
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH
OCCE4001 - SOFTWARE TESTING

Create app.py, Jenkinsfile, test_app.py, and Dockerfile and push them to GitHub.
for ex: demo Project

app.py

```
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b  
  
def multiply(a, b):  
    return a * b  
  
if __name__ == "__main__":  
    print("Simple CI/CD Demo App")  
    print("2 + 3 =", add(2, 3))
```

Jenkinsfile

```
pipeline {  
    agent any  
  
    stages {  
        stage('Clone') {  
            steps {  
                echo 'Cloning repository...'  
                checkout scm  
            }  
        }  
  
        stage('Build') {
```

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
CHANDUBHAI S. PATEL INSTITUTE OF TECHNOLOGY
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH
OCCE4001 - SOFTWARE TESTING

```
steps {
    echo 'Installing dependencies...'
    sh 'pip3 install --break-system-packages -r requirements.txt'
}

stage('Test') {
    steps {
        echo 'Running test cases...'
        sh 'python3 -m pytest test_app.py -v'
    }
}

post {
    success {
        echo 'Pipeline finished successfully!'
    }
    failure {
        echo 'Pipeline failed. Check console output above.'
    }
}
```

test_app.py

```
from app import add, subtract, multiply

def test_add():
    assert add(2, 3) == 5
```

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
CHANDUBHAI S. PATEL INSTITUTE OF TECHNOLOGY
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH
OCCE4001 - SOFTWARE TESTING

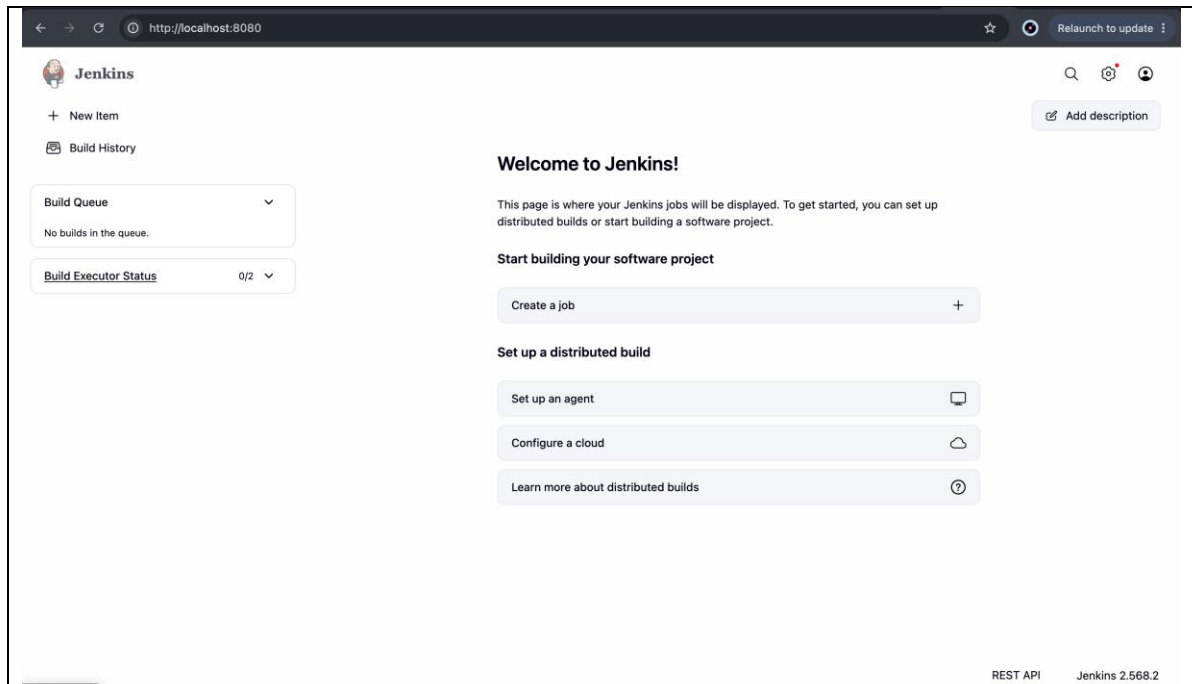
```
def test_subtract():  
    assert subtract(5, 2) == 3  
  
def test_multiply():  
    assert multiply(3, 3) == 9  
  
if __name__ == "__main__":  
    test_add()  
    test_subtract()  
    test_multiply()  
    print("All tests passed!")
```

Dockerfile

```
FROM python:3.9-slim  
WORKDIR /app  
COPY . .  
RUN pip install --break-system-packages -r requirements.txt  
CMD ["python3", "app.py"]
```

Now open jenkins <http://localhost:8081/>

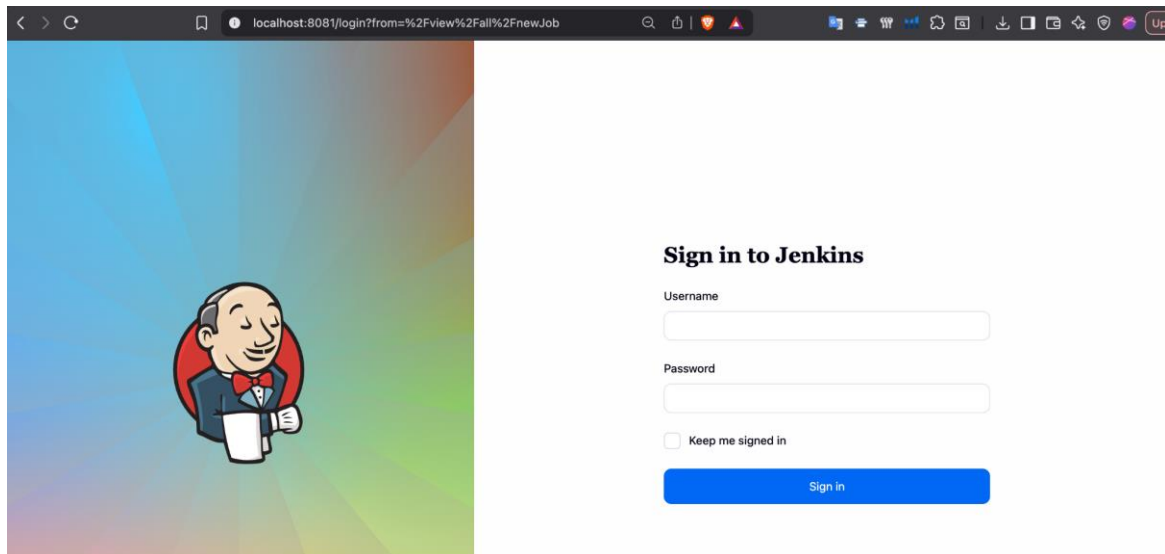
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
CHANDUBHAI S. PATEL INSTITUTE OF TECHNOLOGY
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH
OCCE4001 - SOFTWARE TESTING



localhost:8080

REST API Jenkins 2.568.2

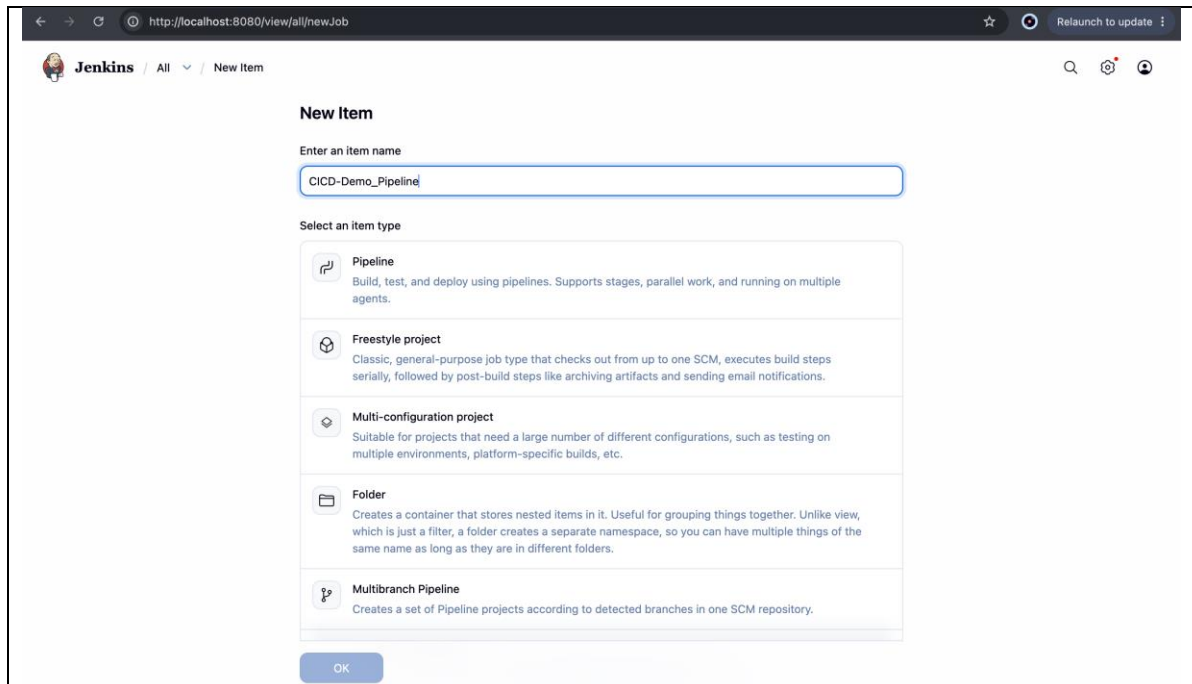
Click "New item" left side



Step 9: Create Pipeline Job

Create a new Jenkins Pipeline project and configure it.

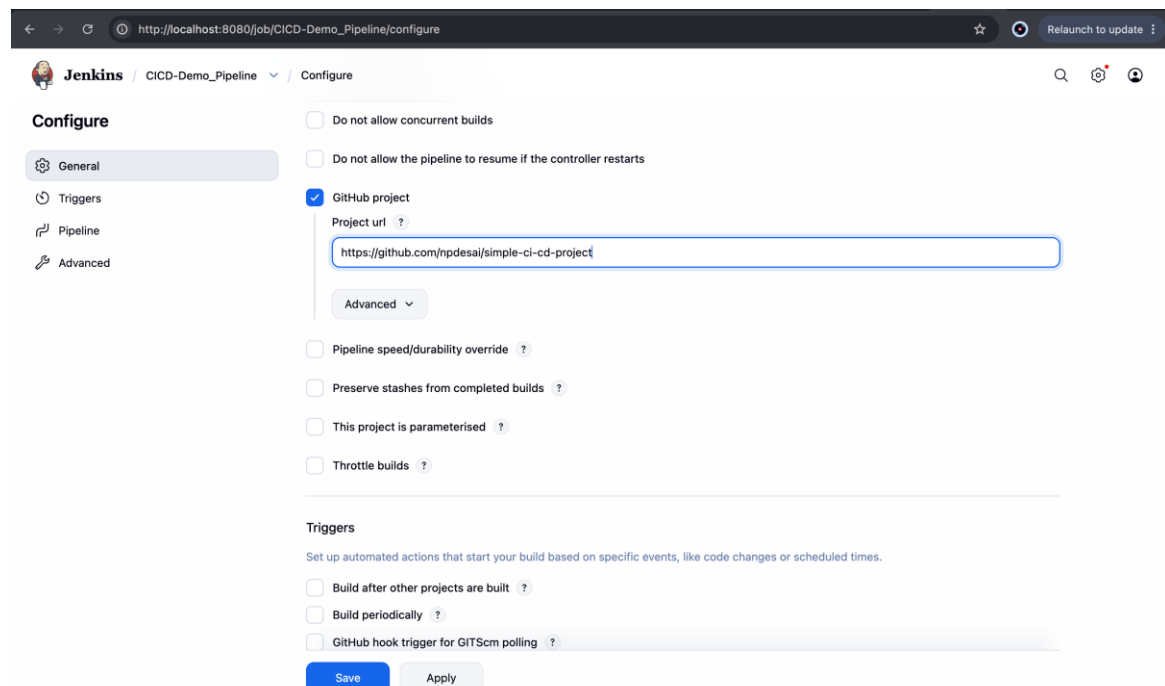
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
CHANDUBHAI S. PATEL INSTITUTE OF TECHNOLOGY
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH
OCCE4001 - SOFTWARE TESTING



The screenshot shows the Jenkins 'New Item' page in a web browser. The address bar displays 'http://localhost:8080/view/all/newJob'. The Jenkins logo and 'All / New Item' breadcrumb are at the top. The main heading is 'New Item'. Below it, there is a text input field labeled 'Enter an item name' containing 'CICD-Demo_Pipeline'. Underneath, a section titled 'Select an item type' lists five options: 'Pipeline' (with a description: 'Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.'), 'Freestyle project' (description: 'Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.'), 'Multi-configuration project' (description: 'Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.'), 'Folder' (description: 'Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.'), and 'Multibranch Pipeline' (description: 'Creates a set of Pipeline projects according to detected branches in one SCM repository.'). An 'OK' button is at the bottom.

Step 10: Configure Git Repository

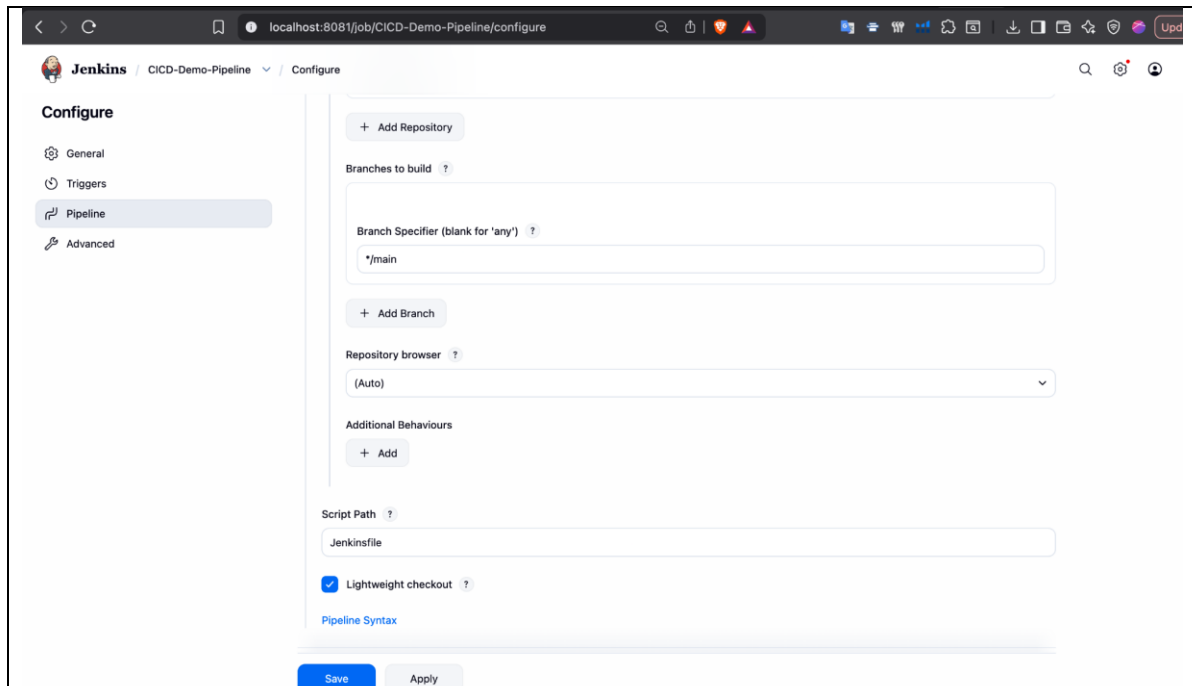
Configure Pipeline Script from SCM and connect the GitHub repository.



The screenshot shows the Jenkins 'Configure' page for the 'CICD-Demo_Pipeline' job. The address bar displays 'http://localhost:8080/job/CICD-Demo_Pipeline/configure'. The left sidebar has 'Configure' at the top, followed by 'General' (selected), 'Triggers', 'Pipeline', and 'Advanced'. The main area shows configuration options. Under 'General', there are checkboxes for 'Do not allow concurrent builds', 'Do not allow the pipeline to resume if the controller restarts', and 'GitHub project' (which is checked). Below 'GitHub project' is a 'Project url' field containing 'https://github.com/npdesai/simple-ci-cd-project'. There is an 'Advanced' dropdown button. Below these are several unchecked checkboxes: 'Pipeline speed/durability override', 'Preserve stashes from completed builds', 'This project is parameterised', and 'Throttle builds'. The 'Triggers' section has a description 'Set up automated actions that start your build based on specific events, like code changes or scheduled times.' and three unchecked checkboxes: 'Build after other projects are built', 'Build periodically', and 'GitHub hook trigger for GITScm polling'. At the bottom are 'Save' and 'Apply' buttons.

Scroll down and select “Pipeline script from SCM”. then SCM select “Git” then enter Github repository URL and in Branch “ */main ”

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
CHANDUBHAI S. PATEL INSTITUTE OF TECHNOLOGY
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH
OCCE4001 - SOFTWARE TESTING



Final click on save

Update jenkins

```
docker exec -u root jenkins bash -c "apt-get update"
```

Step 11: Install Python in Jenkins

Install Python and pip inside the Jenkins container and verify the installation.

Install python:

```
docker exec -u root jenkins bash -c "apt-get install -y python3 python3-pip"
```

Verify installation :

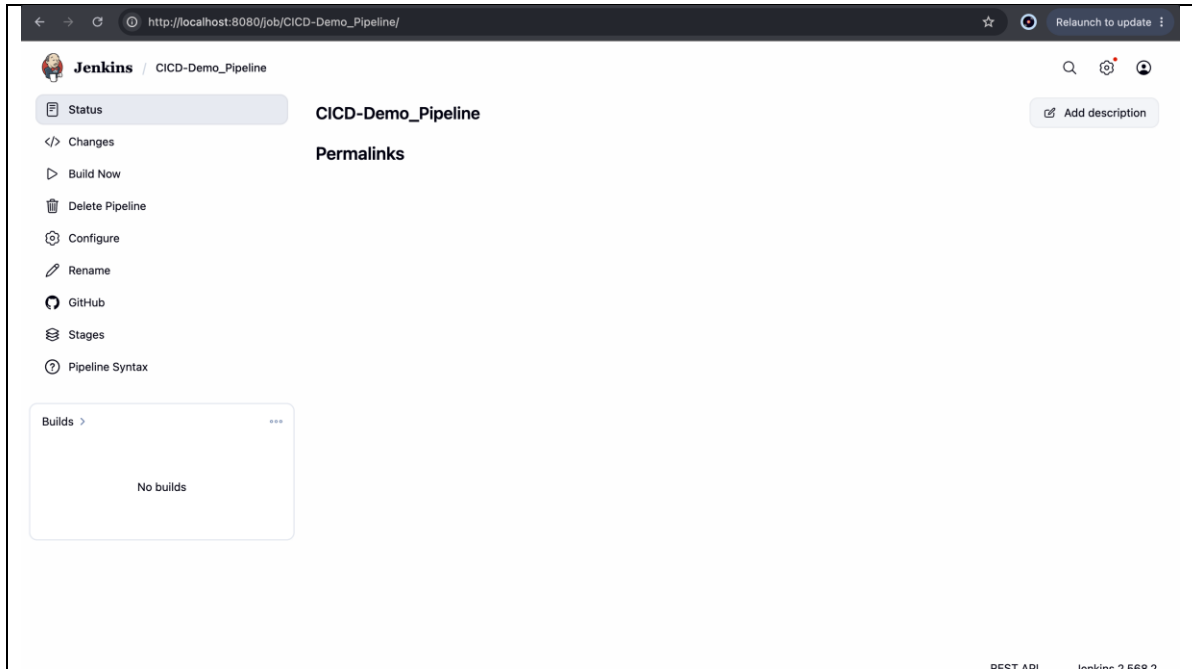
```
docker exec jenkins python3 --version
```

```
docker exec jenkins pip3 --version
```

Step 12: Build the Pipeline

Run Build Now and monitor the pipeline execution.

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
CHANDUBHAI S. PATEL INSTITUTE OF TECHNOLOGY
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH
OCCE4001 - SOFTWARE TESTING

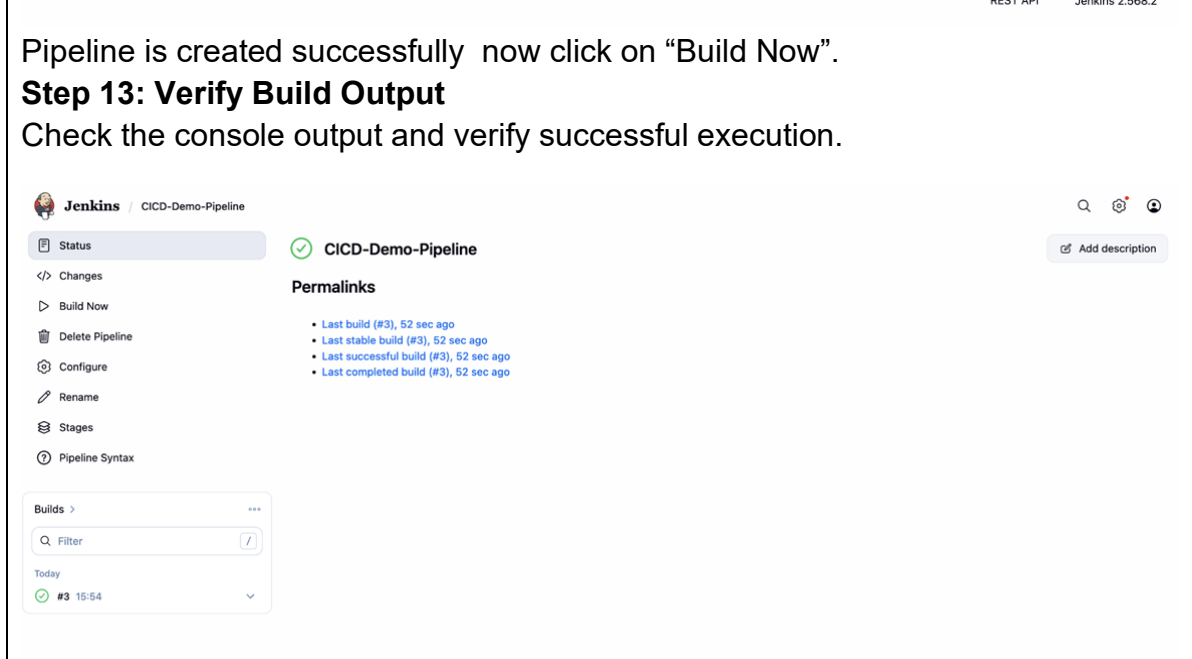


The screenshot shows the Jenkins web interface for a pipeline named "CICD-Demo_Pipeline". The left sidebar contains navigation links: Status (selected), Changes, Build Now, Delete Pipeline, Configure, Rename, GitHub, Stages, and Pipeline Syntax. The main area displays the pipeline name and a "Permalinks" section. Below this, a "Builds" section shows "No builds". At the bottom right, it indicates "REST API" and "Jenkins 2.568.2".

Pipeline is created successfully now click on “Build Now”.

Step 13: Verify Build Output

Check the console output and verify successful execution.



The screenshot shows the same Jenkins interface after a successful build. The "Status" tab is still selected. The "Permalinks" section now lists four links: "Last build (#3), 52 sec ago", "Last stable build (#3), 52 sec ago", "Last successful build (#3), 52 sec ago", and "Last completed build (#3), 52 sec ago". The "Builds" section shows a single build, "#3", with a green checkmark and the time "15:54".

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
CHANDUBHAI S. PATEL INSTITUTE OF TECHNOLOGY
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH
OCCE4001 - SOFTWARE TESTING

```
Jenkins / CICD-Demo-Pipeline #3

Successfully installed iniconfig-2.3.0 pluggy-1.6.0 pygments-2.20.0 pytest-9.1.1
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Running test cases...
[Pipeline] sh
+ python3 -m pytest test_app.py -v
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /var/jenkins_home/workspace/CICD-Demo-Pipeline
collecting ... collected 3 items

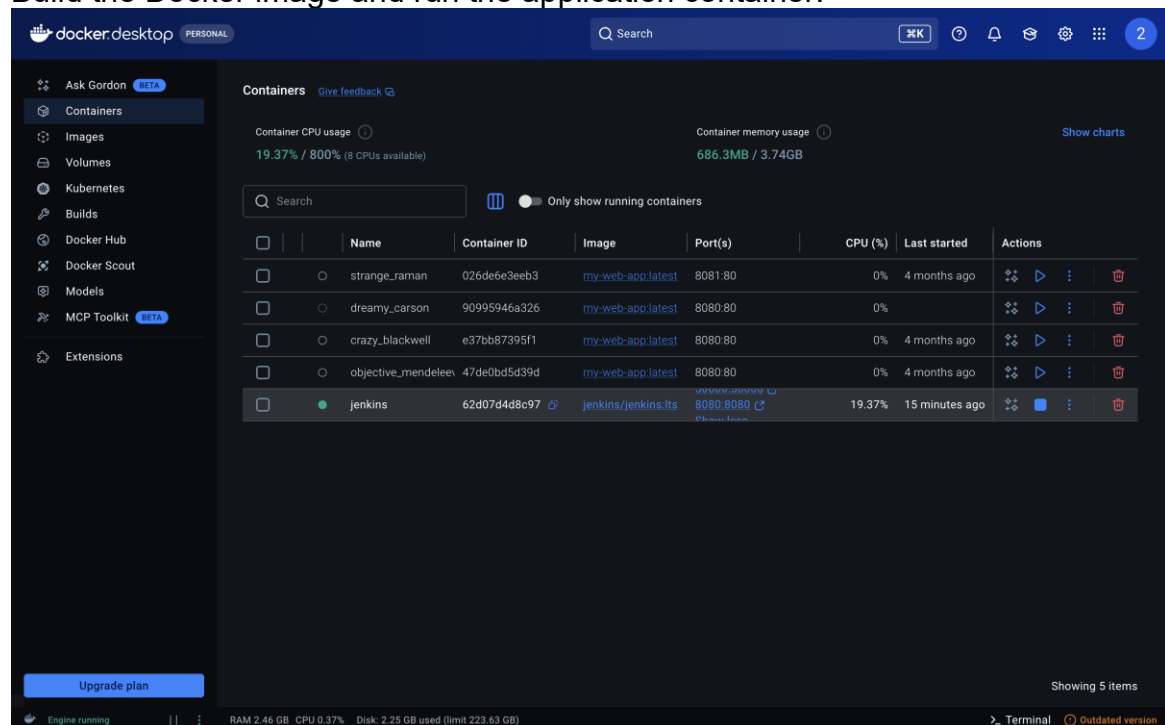
test_app.py::test_add PASSED [ 33%]
test_app.py::test_subtract PASSED [ 66%]
test_app.py::test_multiply PASSED [100%]

===== 3 passed in 0.00s =====
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline finished successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

In console Output, you can able to see the Success message.

Step 14: Build and Run Docker Image

Build the Docker image and run the application container.



Now run the app in a docker container.

cd Desktop/Script4-CICD

docker build -t myapp .

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
CHANDUBHAI S. PATEL INSTITUTE OF TECHNOLOGY
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH
OCCE4001 - SOFTWARE TESTING

<code>docker run myapp</code>
Reference Youtube Video: https://www.youtube.com/watch?v=PKcGy9oPVXg

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
CHANDUBHAI S. PATEL INSTITUTE OF TECHNOLOGY
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH
OCCE4001 - SOFTWARE TESTING